



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 7: Suite de Pruebas Unitarias e Integración de Cero Tolerancia a Defectos para la Plataforma "ExpresoFast"

Metadatos

- **Tiempo Estimado:** 6 horas (trabajo autónomo individual)

Herramientas Requeridas:

- Java 21 (o versión local instalada) con Maven.
- Spring Boot 3.x con Spring Data JPA y Spring Security.
- JUnit 5 Jupiter, Mockito 5.x y MockMvc.
- Plugin JaCoCo para análisis de cobertura de código.

- Microsoft SQL Server Developer Edition.
- Git y Cuenta activa de GitHub.

Metas de Aprendizaje:

1. Diseñar e implementar una suite completa de pruebas unitarias aisladas para la capa de lógica de negocio (`EnvioService`, `VehiculoService`, `EmpresaLogisticaService`) mediante JUnit 5 y Mockito.
2. Construir pruebas de corte de controladores (*Controller Slices*) con `@WebMvcTest` y `MockMvc` para validar los códigos de estado HTTP (200, 201, 400, 403, 404) y la estructura de las respuestas JSON.
3. Implementar pruebas parametrizadas con `@ParameterizedTest` para validar reglas de negocio complejas y tarifas dinámicas de envío.
4. Configurar y ejecutar el plugin `jacoco-maven-plugin` exigiendo un umbral mínimo del **85% de cobertura de instrucciones** en el dominio de servicios.
5. Versionar la suite de pruebas en GitHub utilizando buenas prácticas de integración continua y documentar la evidencia en Mediación Virtual.

Introducción y Caso de Estudio (Tercera Parte)

En las etapas previas del proyecto **ExpresoFast** (Laboratorios 5 y 6), usted diseñó el modelo relacional relacional, optimizó las consultas JPQL con `JOIN FETCH`, implementó la arquitectura de DTOs, la seguridad basada en JWT y el control de acceso con RBAC (`ROLE_ADMIN`, `ROLE_OPERADOR`, `ROLE_CONDUCTOR`).

Para esta **Tercera Parte**, la Gerencia de Aseguramiento de Calidad (QA) de ExpresoFast ha establecido una directiva de **"Cero Tolerancia a Defectos"** previa a la puesta en producción del sistema. Para dar cumplimiento a esta norma de ingeniería de software, su equipo debe certificar la plataforma mediante una suite automatizada de pruebas unitarias y de integración en capa web que garantice que ningún cambio futuro introduzca regresiones.

Requerimientos Técnicos del Laboratorio

1. Configuración de Entorno de Pruebas y Cobertura en Maven (pom.xml)

Usted debe configurar su proyecto Spring Boot para soportar la ejecución automatizada de pruebas y el reporte de cobertura mediante Maven:

- Asegurar la inclusión de la dependencia `spring-boot-starter-test`.
- Configurar el plugin `maven-surefire-plugin` en su versión 3.2.5 para la ejecución fluida de pruebas JUnit 5.
- Configurar el plugin `jacoco-maven-plugin` en su versión 0.8.11 definiendo las metas `prepare-agent` y `report` durante la fase `test`.
- Establecer una regla de verificación de cobertura (`jacoco:check`) que falle la compilación si la cobertura de instrucciones en `com.expresofast.service` es menor al **85%**.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
    <execution>
      <id>check</id>
      <goals>
        <goal>check</goal>
      </goals>
      <configuration>
        <rules>
          <rule>
```

```

        <element>PACKAGE</element>
        <includes>
            <include>com.expresofast.service</include>
        </includes>
        <limits>
            <limit>
                <counter>INSTRUCTION</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.85</minimum>
            </limit>
        </limits>
    </rule>
</rules>
</configuration>
</execution>
</executions>
</plugin>

```

2. Pruebas Unitarias de Servicios de Negocio

(@ExtendWith(MockitoExtension.class))

Deberá escribir clases de prueba para los servicios centrales de la aplicación, aislando completamente las dependencias de base de datos mediante mocks de Mockito (@Mock e @InjectMocks):

- [] **EnvioServiceTest:**
- `crearEnvio_DatosValidos_RetornaEnvioDTO`: Verifica la creación exitosa de un envío y la asignación del estado inicial `PENDIENTE`.
- `crearEnvio_VehiculoSinCapacidad_LanzaExcepcion`: Verifica que se lance `CapacidadExcedidaException` al intentar asignar un paquete que supere el peso máximo permitido del vehículo.
- `actualizarEstado_TransicionInvalida_LanzaExcepcion`: Verifica que un envío en estado `ENTREGADO` no pueda cambiar retroactivamente a `EN_TRANSITO`.
- `cancelarEnvio_EnvioEnTransito_LanzaExcepcion`: Asegura que no se puedan cancelar envíos que ya se encuentran en ruta.

[] **VehiculoServiceTest:**

- `registrarVehiculo_PlacaDuplicada_LanzaExcepcion`: Simula la existencia de una placa en `VehiculoRepository.existsByPlaca()` y verifica la captura de `DuplicateResourceException`.
- `asignarConductor_ConductorInactivo_LanzaExcepcion`: Valida que no se pueda asignar un conductor inactivo a un vehículo.

3. Pruebas de Capa de Controladores REST (@WebMvcTest y MockMvc)

Deberá validar el contrato HTTP de sus controladores REST mediante pruebas de corte de controlador (*Slice Testing*), deshabilitando temporalmente o simulando los filtros de seguridad:

- `[] EnvioControllerTest`:
 - `GET /api/envios/{id}` (Exitoso): Retorna HTTP 200 OK y valida los atributos JSON del envío mediante `jsonPath("$.codigoRastreo")`.
 - `GET /api/envios/{id}` (No encontrado): Simula que el servicio lanza `ResourceNotFoundException` y verifica que el controlador responda HTTP 404 Not Found bajo el formato RFC 7807.

`POST /api/envios` (Payload Inválido): Envía un objeto JSON con campos requeridos nulos o vacíos (`@NotBlank`, `@Positive`) y verifica que retorne HTTP 400 Bad Request con la lista de errores de validación.

`[] AuthControllerTest`:

- `POST /api/auth/login` (Credenciales Correctas): Retorna HTTP 200 OK con el token JWT de acceso en el cuerpo de la respuesta JSON.
- `POST /api/auth/login` (Credenciales Incorrectas): Retorna HTTP 401 Unauthorized cuando la autenticación falla.

4. Pruebas Parametrizadas de Cálculo de Tarifas de Envío

Implemente pruebas parametrizadas utilizando `@ParameterizedTest` y `@CsvSource` para certificar la matriz de cálculo de fletes según distancia (km) y peso (kg):

```
@ParameterizedTest
@CsvSource({
    "5.0, 10.0, 2500.0",
    "15.0, 50.0, 7500.0",
    "100.0, 2.5, 12000.0"
})
@DisplayName("Debe calcular la tarifa correcta segun peso y distancia")
void calcularTarifa_CasosVariados_CalculaCorrectamente(
    double pesoKg, double distanciaKm, double tarifaEsperada) {

    double tarifaCalculada = envioService.calcularTarifa(
        pesoKg, distanciaKm
    );
    assertEquals(tarifaEsperada, tarifaCalculada, 0.01);
}
```

5. Certificación de Cobertura JaCoCo

Una vez implementadas todas las clases de prueba, ejecute el comando completo de construcción y verificación en Maven:

```
mvn clean verify
```

Verifique que: 1. El proceso finalice con la leyenda `BUILD SUCCESS`. 2. El archivo de reporte visual `target/site/jacoco/index.html` refleje una cobertura superior al **85%** en las clases del paquete de servicio. 3. Adjunte capturas de pantalla de la interfaz web del reporte JaCoCo en la documentación de entrega.

Entregables y Modalidad de Entrega

Repositorio en GitHub:

- Suba el código fuente de la suite de pruebas a la rama `main` de su repositorio de GitHub.

- Incluya un archivo `README.md` actualizado en la raíz indicando las instrucciones para ejecutar las pruebas (`mvn clean test`) y ver el reporte de cobertura.

Entrega en Mediación Virtual:

- Suba un documento PDF o Markdown comprimido que incluya:
 - Enlace público o acceso concedido al repositorio de GitHub.
 - Captura de pantalla de la terminal mostrando la ejecución exitosa de `mvn clean verify`.
 - Capturas de pantalla del reporte HTML de cobertura JaCoCo (`target/site/jacoco/index.html`).

Rúbrica de Evaluación (Total: 100 Puntos)

Criterio de Evaluación	Puntuación	Descripción
Configuración de Maven y JaCoCo	10 pts	Integración correcta de <code>spring-boot-starter-test</code> , <code>maven-surefire-plugin</code> y <code>jacoco-maven-plugin</code> con umbral del 85%.
Pruebas Unitarias de Servicios	30 pts	Cobertura completa de casos de éxito y de excepción en <code>EnvioServiceTest</code> , <code>VehiculoServiceTest</code> y <code>EmpresaLogisticaServiceTest</code> con Mockito.

Pruebas de Controladores con MockMvc	25 pts	Pruebas de corte de controlador en <code>EnvioControllerTest</code> y <code>AuthControllerTest</code> verificando códigos HTTP (200, 201, 400, 404) y <code>jsonPath</code> .
Pruebas Parametrizadas y Excepciones	15 pts	Uso de <code>@ParameterizedTest</code> con <code>@CsvSource</code> y aserciones <code>assertThrows</code> para reglas complejas de cálculo de tarifas y validaciones.
Certificación de Cobertura JaCoCo ($\geq 85\%$)	10 pts	Cumplimiento estricto del umbral del 85% de cobertura de instrucciones en la capa de servicios evidenciado en el reporte final.
GitHub, Documentación y Entrega	10 pts	Estructura profesional del repositorio Git, commits semánticos, instrucciones claras en <code>README.md</code> y reporte entregado a tiempo.